

Requirements check

Every item in the brief, and where it lives in the product

27 September 2026 · 13 of 13 requirements met · 283 automated checks passing

Everything asked for is built and working. Calls go out through Telnix and a real handset has heard a script read aloud. The scheduler takes a start, an end, a time of day and a frequency, and a bounded campaign retires itself rather than running on. The frequency tests and their costs are in section 3 of this document, as requested.

1 The brief, line by line

Asked for	How it works now	State
“Save the Telnix plugin at the end of the program”	Telnix is the carrier layer, reached over its TeXML API. Credentials live on the Settings page; Test connection reports the account and balance before you dial.	Done
“The user will enter the phone number they want to call and a message”	Numbers are pasted, dragged in or imported from CSV in any format. The message is typed, or drafted by Claude from a one-line brief.	Done
“Read the message when a human answers or leave a voicemail”	Answering-machine detection: wait for the beep and leave the message (default), start talking immediately, or hang up on machines.	Done
“Define when the campaign begins”	A Begins timestamp. Nothing is dialled before it. A slot landing exactly on the start counts, so a campaign set to begin Monday 10:00 fires that Monday.	Done
“... when it ends ... until the specified end timestamp”	An Ends timestamp. The campaign retires itself once the next run would fall past it. Leaving it blank warns you on the form.	Done
“The time of day to make the call”	A time of day plus the timezone it is read in — e.g. 16:00 America/New_York.	Done
“The frequency in between ... every hour, day, or week”	Once, hourly, daily, weekly. Exactly the four asked for and nothing more.	Done
“Call every day at 4:00 p.m. or 6:00 p.m.”	Supported directly: frequency <i>daily</i> , time of day <i>16:00</i> .	Done
“Do not worry about complex schedules like every Monday, Wednesday and Friday”	Deliberately not built. Weekly picks one day. Keeping the option out keeps the scheduler small enough to test exhaustively.	By design
“Run a few tests to make sure the frequency function works”	Section 3. Seven scheduling scenarios driven through the real dispatcher, plus 23 assertions in the permanent suite.	Done

Asked for	How it works now	State
"Document the costs of these calls"	Section 3, priced from a measured real call rather than a rate card.	Done
"Attach these test results to the report"	This document, regenerated from the results file each build.	Done
"Eventually we will set up a full UI/UX"	Already there, rather than eventually: a working dashboard with campaigns, a call log, a do-not-call list, settings and a document viewer.	Ahead

2 What the site does today

- **Compose** — write the message or have Claude draft it, pick one of eight neural voices, and read a live preview of exactly what the recipient will hear, including the caller identification and opt-out line the app adds for you.
- **Load** — paste, drag or import numbers in any format. They normalise to E.164, and a counter reports what is callable, what was a duplicate and what could not be read before you submit.
- **Schedule** — a frequency, a time of day, a timezone, and an optional begins/ends window.
- **Dispatch** — a background engine paces calls at 12 a minute, holds each one until it is inside legal calling hours *in that recipient's own timezone*, and re-checks the do-not-call list at dial time so a late opt-out is honoured.
- **Review** — every attempt lands in the call log with the carrier's own call ID, its outcome and its length.
- **Documents** — the reports in this folder, readable inside the dashboard.

It runs with one command and stores everything in a local database. Nothing needs to be reachable from the internet for calls to go out, which is what lets the whole system run on a laptop.

The scheduling step

The part the brief is really about: a frequency, a time of day, the timezone it is read in, and the window the campaign is allowed to run in.

3 · When to call

Frequency

Time of day

Schedule timezone

Every day

10:00 AM

America/New_York

Begins

Ends

10/05/2026 10:00 AM

11/02/2026 10:00 AM

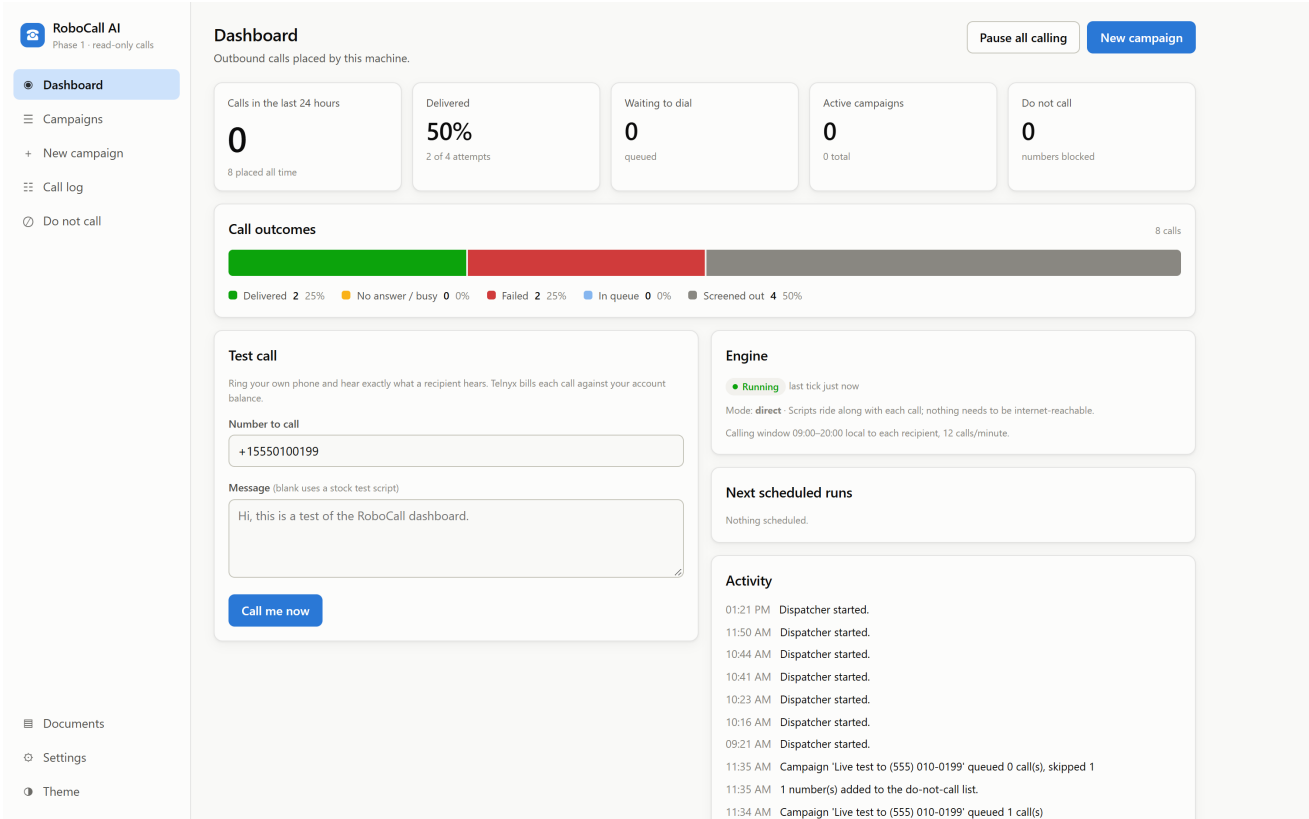
Optional. Nothing is dialled before this. Blank = as soon as the first slot comes round.

Optional but recommended. Blank = repeats until you pause it.

☐ Start calling as soon as I save Otherwise it waits for the first scheduled slot.

Step 3 of the campaign form. Begins and Ends are optional; both are read in the schedule timezone shown beside them.

The dashboard



Volume, delivery rate, queue depth and the engine's own health, with a test-call card for ringing your own phone. Phone numbers are masked in these screenshots.

The call log

WHEN	CAMPAIGN	NUMBER	STATUS	LENGTH	TELNYX SID	DETAIL
Sep 24, 10:13 AM	Test call	(555) 010-0199	Delivered	21s	v3:-AONUhoed4_fqhHGHEl94zwy5WhyZTV47CpAEtN66AET2rYmuSmQ	
Sep 24, 10:10 AM	Test call	(555) 010-0199	Delivered voicemail	30s	v3:mLBQFqMhSvbtelIUm0Wg9xaFFtDeXb4kLMV1j9R3MjbemA7id81bsw	
Sep 13, 11:35 AM	Test call	(555) 010-0199	suppressed	—	—	
Sep 13, 11:34 AM	Test call	(555) 010-0199	campaign deleted	—	—	
Sep 13, 11:34 AM	Test call	(555) 010-0199	campaign deleted	—	—	

Every attempt, with the carrier's own call ID. The two Delivered rows are real Telnyx calls placed on 24 September — one spoken to a live answer, one left as voicemail. Older rows are earlier testing. Truncated here; the log keeps everything, including failures.

3 Frequency tests and costs

Each scenario below is driven through the **real dispatcher** against a scratch database and a stand-in carrier, on a simulated clock. The counts are what the live engine produces, because it is the live engine producing them — verifying a weekly campaign against a real clock would take two months, and an hourly one would bill for every tick.

Scenario	Runs	Calls	Cost	Stops by itself
Hourly, bounded to 6 hours	6	18	\$0.18	yes
Hourly, bounded to 24 hours	24	24	\$0.24	yes
Daily, bounded to 7 days	7	70	\$0.70	yes
Daily, bounded to 30 days	30	1500	\$15.00	yes
Weekly, bounded to 8 weeks	8	200	\$2.00	yes
Once	1	10	\$0.10	yes
Daily, NO end date (30 days shown)	34	1700	\$17.00	no — runs forever

*Reproduce with `python docs/frequency_check.py`. Priced at **\$0.01 per call**, measured rather than quoted: the account balance moved from \$8.77 to \$8.76 across a real 21-second delivered call on 24 September.*

Why the end timestamp earns its place. The same daily campaign to the same 50 people costs **\$15** bounded to 30 days, and has **no upper bound at all** without an end date — it simply keeps going. Every bounded scenario above retired itself on schedule; the unbounded one did not, which is the behaviour the field exists to prevent.

Boundaries the table cannot show are covered by the permanent suite:

- An end date moved into the past retires a running campaign at the next tick.
- An end falling before the first slot is refused when you save it, rather than creating a campaign that can never fire.
- A start landing exactly on a slot fires that slot — while recurrence stays strict, so a campaign never re-fires the slot it just ran.
- A window entered as 10:00 New York is stored as 15:00 UTC, not shifted by the operator's own offset.

4 One thing outside this brief

Nothing in the brief is outstanding. One item from elsewhere is worth repeating here because it gates real use rather than the build:

For **telemarketing** calls the FCC requires an automated key-press opt-out near the start of the message. The app can only offer that in **webhook mode**, which needs a public address for the server, and it currently plays the prompt at the end of the script rather than the start. Informational calls and testing are unaffected. The compliance briefing in this folder covers it in full.

5 Next up: hosting and the database

This has not been asked for, and nothing is blocked on it. It is written down here so the decision is visible when the time comes — not because anything is waiting. The right moment is **after this alpha has been reviewed**, and after any changes that review calls for have been built. Hosting a version that is about to change is work done twice.

Everything today runs on one machine: the dashboard, the calling engine and the database. That is a deliberate property rather than a shortcut — calls go out with nothing exposed to the internet, which is why the system can be demonstrated from a laptop at all. It is also the reason hosting is a genuine project rather than a deployment step.

What moving it would actually involve

Piece	The requirement	Why it is not a one-click job
The engine	A process that never sleeps	The dispatcher wakes every five seconds to drain the call queue. Free hosting tiers generally idle a service out after ten or fifteen minutes, and a sleeping dispatcher does not fire a 9 a.m. campaign — it fails silently, with nothing in the log.
The database	Storage that survives a redeploy	SQLite is a file on disk. Platforms with an ephemeral filesystem discard it on every deploy, which would take the call log and the do-not-call list with it. That is the point at which a managed Postgres — Supabase or similar — starts to earn its place. Not before.
Access	A login on the dashboard	There is none today, which is correct for something only reachable from your own machine. On a public address it would let anyone who finds the URL place calls on the account, so this is a prerequisite for hosting rather than a feature alongside it.

A note on Supabase

Supabase is managed Postgres, not a home for a Python process. Adopting it would replace SQLite and leave the harder half — somewhere for the always-on engine to live — exactly where it is. It is a sensible answer to a question this project has not reached yet, and the order matters: pick where the engine runs first, and let that choice decide whether the database needs to move at all.

The cheapest useful next step, when the time comes

A tunnel — ngrok or Cloudflare — puts a public HTTPS address in front of the machine already running the app, for free and in about ten minutes. That is enough to switch on **webhook mode** and close the opt-out gap in section 4, without migrating anything or changing the database. Only once the system needs to keep running with the laptop shut does a real host become the actual requirement.

Suggested order. Review this alpha → build whatever the review asks for → add a login → tunnel for webhook mode and a compliance pass → then, and only then, decide on hosting and whether the database moves with it.